

Unit 10

Arrays and Matrix Multiplications

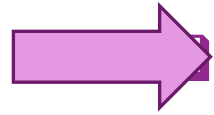
ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

- ❑ Estimate compute and memory operations for basic matrix operations
 - Determine which operations are **compute** or **data-limited**
- ❑ Describe how to store and address matrices in sequential memory
 - **Row-major** and **column-major**
- ❑ Implement simple baseline multiplication with loop unrolling
 - Implement using Vitis pragmas
 - Estimate cost
- ❑ Recognize high **fanout** computations
- ❑ Implement and describe cycle-to-cycle behavior of **systolic arrays**
- ❑ Implement and estimate timing for **tiled** matrix multiplication

Outline



Matrix Multiplications

- Hardware Implementation via Broadcast
- Systolic Arrays
- Tiling for Large Matrices

What is a Matrix?

- ❑ An array of numbers
- ❑ Rows x columns
- ❑ Many applications
- ❑ Data matrix
 - Ex: Each row is a set of values of a sample
- ❑ Image
 - Each entry is a pixel
- ❑ Time series
 - Each row is a time step
 - Each column is some attribute that changes over time

$A_{4 \times 5} = 4 \text{ ROWS}$

5 COLUMNS

→	↓	↓	↓	↓	↓
→	-2	7	11	0	3
→	19	22	-8	5	21
→	4	16	24	-12	1
→	31	27	9	43	-18

Matrix Addition and Subtraction

□ $Y = A + B$

□ A and B must be the same size

- Can be rectangular

□ Simply add elements:

- $Y_{ij} = A_{ij} + B_{ij}$

$$\begin{bmatrix} 4 & 8 \\ 3 & 7 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 5 & 2 \end{bmatrix} = \begin{bmatrix} 4+1 & 8+0 \\ 3+5 & 7+2 \end{bmatrix}$$

From Khan Academy.

Excellent review of matrix math if you need it

Matrix Addition Complexity

- ❑ Suppose $A, B \in \mathbb{R}^{M \times N}$
- ❑ Reading $A, B : \frac{MN}{W}$ each matrix
 - W = elements per interface word
- ❑ Computation MN multiplications
- ❑ Writing $A + B : \frac{MN}{W}$
- ❑ Complexity dominated by reading and writing
 - Called **data-limited**
 - Not compute limited
- ❑ Parallelizing additions does not help significantly
 - Unless matrices are already loaded

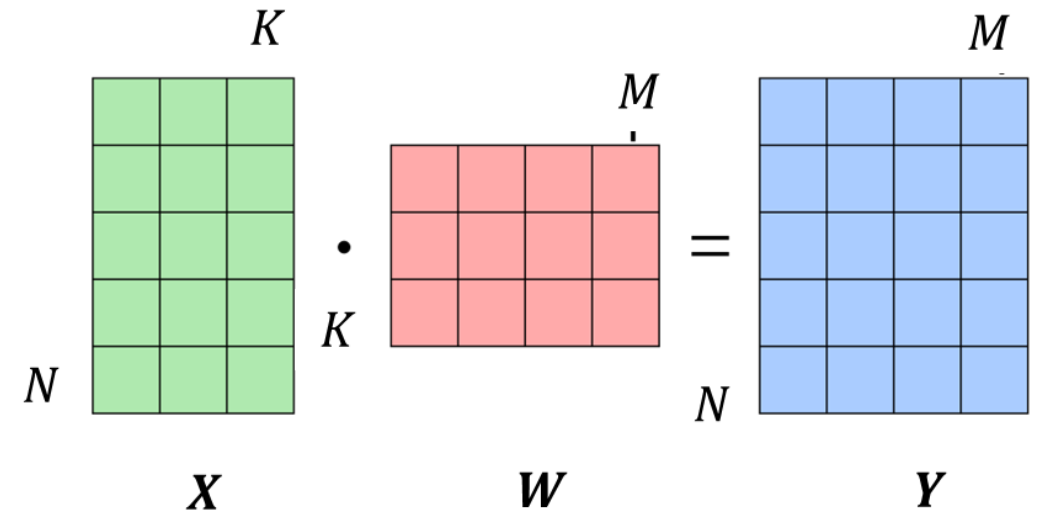
$$A+B = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix}$$

$$A+B = \begin{pmatrix} a_{11}+b_{11} & a_{12}+b_{12} \\ a_{21}+b_{21} & a_{22}+b_{22} \\ a_{31}+b_{31} & a_{32}+b_{32} \end{pmatrix}$$

WWW.ANDREAMININI.ORG

Matrix Multiplication

- ❑ Basic operation in range of fields
- ❑ Machine learning
- ❑ Communications
- ❑ Robotics
- ❑ ...
- ❑ Virtually every mathematical field



Multiplying Two Matrices

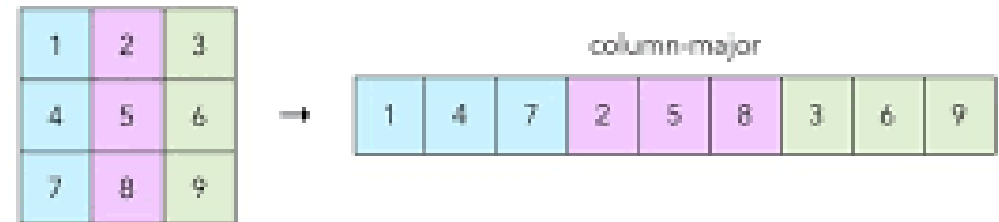
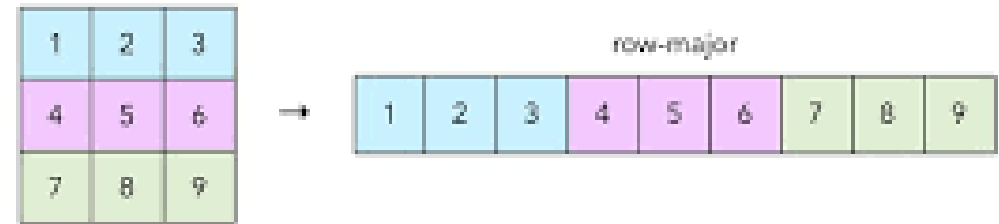
From [excellent Medium post](#)

A Complete Beginners Guide to Matrix Multiplication for Data Science with Python Numpy

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$
$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$
$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

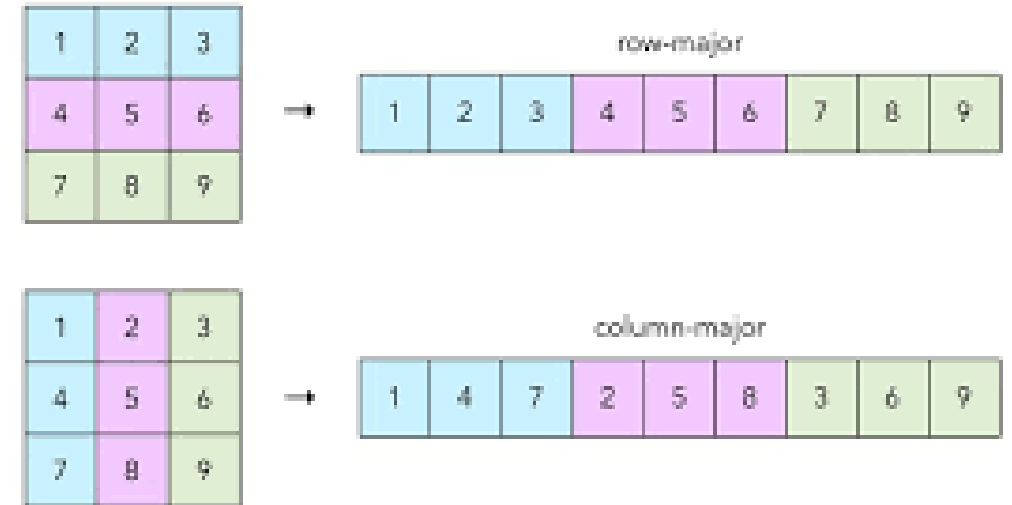
Storing Matrices in Hardware Memory

- ❑ Typically stored in RAM
- ❑ SRAM / BRAM for internal memory
 - Smaller matrices or portions of matrices
- ❑ DRAM for external memory
 - Larger / complete memory
- ❑ Often stored in **row-major order**
 - Store first row $x[0,0], \dots, x[0, m-1]$
 - Store second row $x[1,0], \dots, x[1, m-1]$
 - ...



Row-Major Order in Memory

- ❑ Suppose X is stored in row-major order
- ❑ Consider neighboring elements in the same row
 - Eg. $X[i][n], \dots, X[i][n + k - 1]$
- ❑ These elements are **contiguous**
- ❑ If you have a wide memory bit width
 - Can access neighboring elements in the same column
 - But cannot access elements in neighboring rows



Matrix Multiplication Complexity

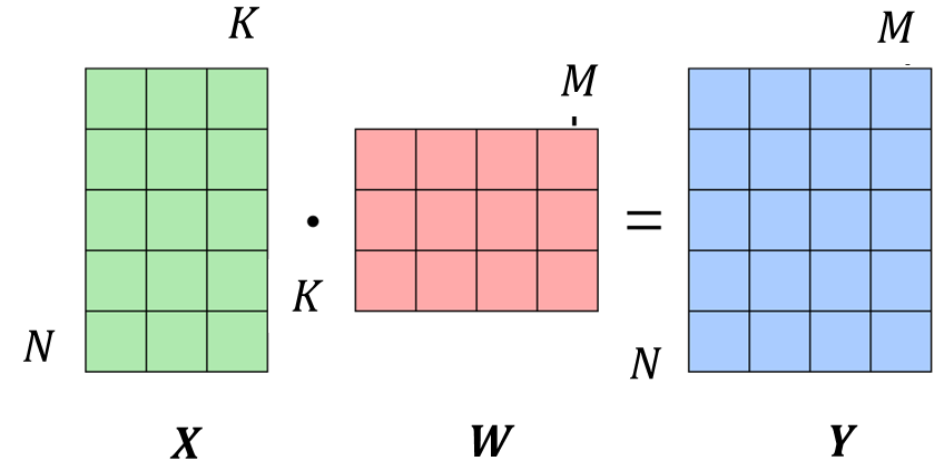
□ Matrix multiplication formula:

$$Y[i, j] = \sum_{k=0}^{N-1} X[i, k]W[k, j]$$

□ Need MNK MAC operations

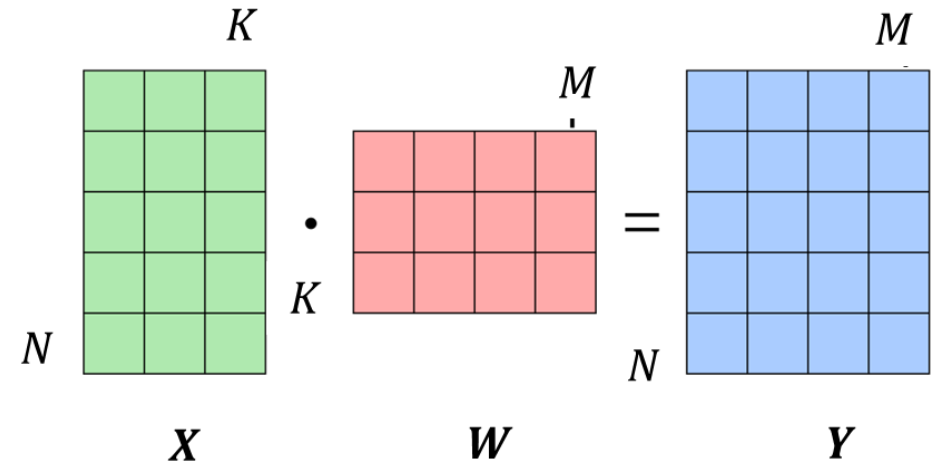
- MAC = multiply and accumulate

□ Complexity grows quickly



Multiplication Memory Accesses

- ❑ Read X and W
 - $NK + KM$ reads
- ❑ Write C
 - NM writes
- ❑ Can read / write B/W words per cycle
 - B = memory bit width
 - W = word bit width
- ❑ Typically compute dominates
- ❑ Thus matrix multiplication needs efficient compute



Sample Problem

Question 2. Multiplication complexity

A hardware module must compute the matrix product $C = AB$, where A is an 1024×16 matrix and B is a 16×8 matrix. Each element is a 32-bit word. Suppose:

- The matrices are stored in a common shared memory with a 64-bit data bitwidth with one read / write per cycle.
- The memory is half-duplex meaning reads and writes cannot occur simultaneously.
- The hardware module has N parallel multipliers, each can compute one multiplication per cycle.

Under these assumptions, answer the following questions:

- a. How many cycles are required to read all elements of matrix A and B ?
- b. How many cycles are required to write all elements of matrix C ?
- c. How many cycles are required to perform all multiplications for the matrix product $C = AB$ as a function of N ?
- d. For what values of N is the multiplication step the bottleneck in the computation, meaning it takes more cycles than the memory read and write steps combined?

Outline

☐ Matrix Multiplications

 ☐ Hardware Implementation via Broadcast

☐ Systolic Arrays

☐ Tiling for Large Matrices

Naïve Implementation in Vitis

```
void matmult_naive() {  
  
    float X[nrow][nw];  
    float W[nw][ncol];  
    float Y[nrow][ncol];  
  
    for (int i = 0; i < nrow; ++i) {  
        for (int j = 0; j < ncol; ++j) {  
            Y[i][j] = 0.0;  
            for (int k = 0; k < nw; ++k) {  
                Y[i][j] += X[i][k] * W[k][j];  
            }  
        }  
    }  
}
```

- ❑ Classic triple nested loop
- ❑ Takes NMK iterations
- ❑ Simple, but slow
- ❑ How can we speed this up?

Problem 1: No Parallelism

```
void matmult_naive() {  
  
    float X[nrow][nw];  
    float W[nw][ncol];  
    float Y[nrow][ncol];  
  
    for (int i = 0; i < nrow; ++i) {  
        for (int j = 0; j < ncol; ++j) {  
            Y[i][j] = 0.0;  
            for (int k = 0; k < nw; ++k) {  
                Y[i][j] += X[i][k] * W[k][j];  
            }  
        }  
    }  
}
```

- ❑ Triple loop requires NMK cycles
- ❑ Only one multiply per cycle
- ❑ Resource efficient
- ❑ But slow

- ❑ We could try to unroll
- ❑ That is, do multiplications in parallel
- ❑ But...

Problem 2: Array Accesses

```
void matmult_naive() {  
  
    float X[nrow][nw];  
    float W[nw][ncol];  
    float Y[nrow][ncol];  
  
    for (int i = 0; i < nrow; ++i) {  
        for (int j = 0; j < ncol; ++j) {  
            Y[i][j] = 0.0;  
            for (int k = 0; k < nw; ++k) {  
                Y[i][j] += X[i][k] * W[k][j];  
            }  
        }  
    }  
}
```

- ❑ Each inner iteration requires
 - Read from X and W
 - Write to Y
- ❑ Access are from random locations
- ❑ Unroll requires parallel accesses

Two Assumptions

□ We can obtain a better multiplication under two key assumptions

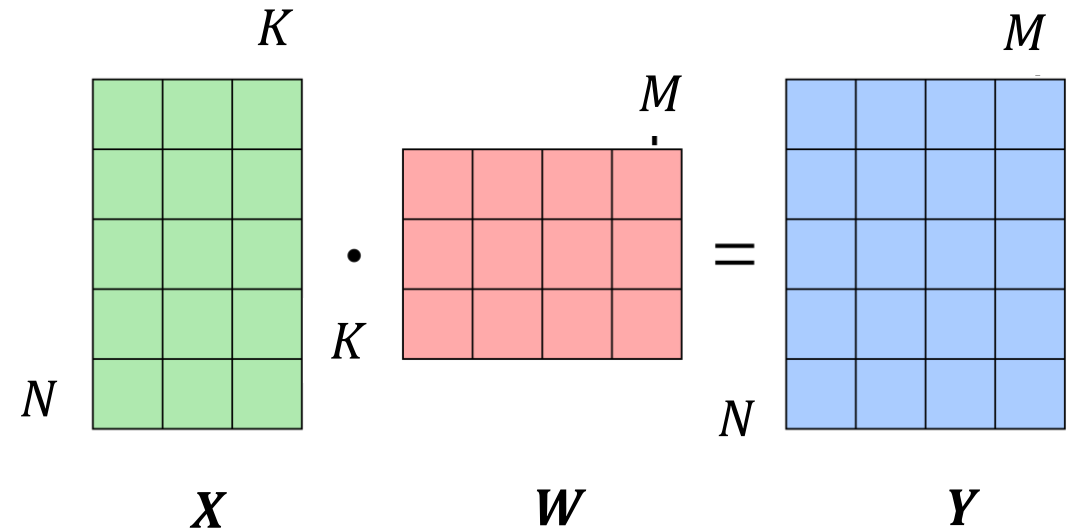
□ **Assumption 1:** W can be stored in register

- That is, fully partitioned memory
- Typically, W must be small

□ **Assumption 2:** X and Y admit row-wise access

- That is, can access all elements of a row in parallel
- Matrices must be in internal memory
- Different columns are in different memory banks
- Or external memory but very wide data width

□ We will relax both assumptions later with tiling



Memory Partitioning in Vitis

```
// Assume X and Y are partitioned along each row, so all elements
// of a row can be accessed in parallel
// These would be filled with some data in a real testbench;
// here we just declare them to show the structure of the code
float X[nrow][nw];
float Y[nrow][ncol];
#pragma HLS ARRAY_PARTITION variable=X type=complete dim=2
#pragma HLS ARRAY_PARTITION variable=Y type=complete dim=2

// Copy W to a local array with full partitioning
float W[nw][ncol];
float Wlocal[nw][ncol];
#pragma HLS ARRAY_PARTITION variable=Wlocal type=complete dim=1
#pragma HLS ARRAY_PARTITION variable=Wlocal type=complete dim=2
for (int i = 0; i < nw; ++i) {
    for (int j = 0; j < ncol; ++j) {
        Wlocal[i][j] = W[i][j];
    }
}
```

□ Partition X and Y row-wise

- All elements of each row can be accessed in parallel

□ Fully partition W

- All elements of W can be accessed in parallel

Local Row Buffers

```
// Create local array for each row of X and Y
float Yrow[ncol];
float Xrow[nw];
#pragma HLS ARRAY_PARTITION variable=Yrow type=complete dim=0
#pragma HLS ARRAY_PARTITION variable=Xrow type=complete dim=0
```

- ❑ Create row buffers
 - Store each row of X and Y
- ❑ Fully partition row buffers

Pipeline Loop

- ❑ Three stages
 - ❑ Load row of $X[i, :]$
 - All K loads are in parallel
 - ❑ Compute $Y[i, :]$
 - All KM MACs are in parallel
 - Note double UNROLL pragma
 - ❑ Store row $Y[i, :]$
 - All M are in parallel

```
for (int i = 0; i < nrow; ++i) {
#pragma HLS PIPELINE II=1

    // Copy the current row of X to a local array
    // Done in parallel since the row is fully partitioned
    for (int k = 0; k < nw; ++k) {
#pragma HLS UNROLL
        Xrow[k] = X[i][k];
    }

    // Compute current row of Y
    for (int j = 0; j < ncol; ++j) {
#pragma HLS UNROLL
        Yrow[j] = 0.0;
        for (int k = 0; k < nw; ++k) {
#pragma HLS UNROLL
            Yrow[j] += Xrow[k] * Wlocal[k][j];
        }
        Y[i][j] = Yrow[j];
    }

    // Store the current row of Y back to the output array
    // Done in parallel since the row is fully partitioned
    for (int j = 0; j < ncol; ++j) {
#pragma HLS UNROLL
        Y[i][j] = Yrow[j];
    }
}
}
```

Best Case Complexity

❑ Each pipeline has three stages

- Latency = 3
- $II = 1$

❑ Complexity: Cycles = $N + 2$

❑ Storage:

- X stored in K banks of N each
- Y stored in M banks of M each
- W stored in KM registers

❑ MAC units: KM

Baseline Implementation Summary

- ❑ Conclusion: Baseline implementation achieves

- Dramatic speed up
- Enables highly parallel MACs

- ❑ But there are two problems:

- ❑ Problem 1: Local storage:

- Requires local storage and partitioned arrays
- We will address this with **tiling**

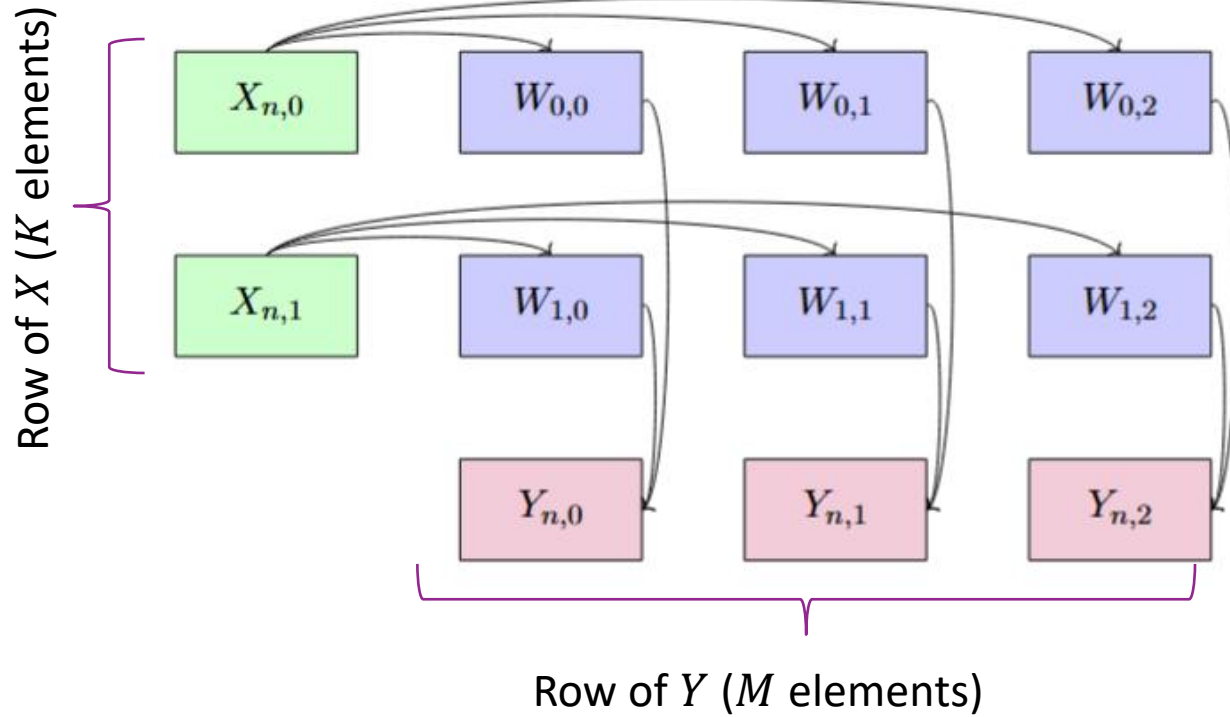
- ❑ Problem 2: Complex **routing**

- We will discuss and address this next

Outline

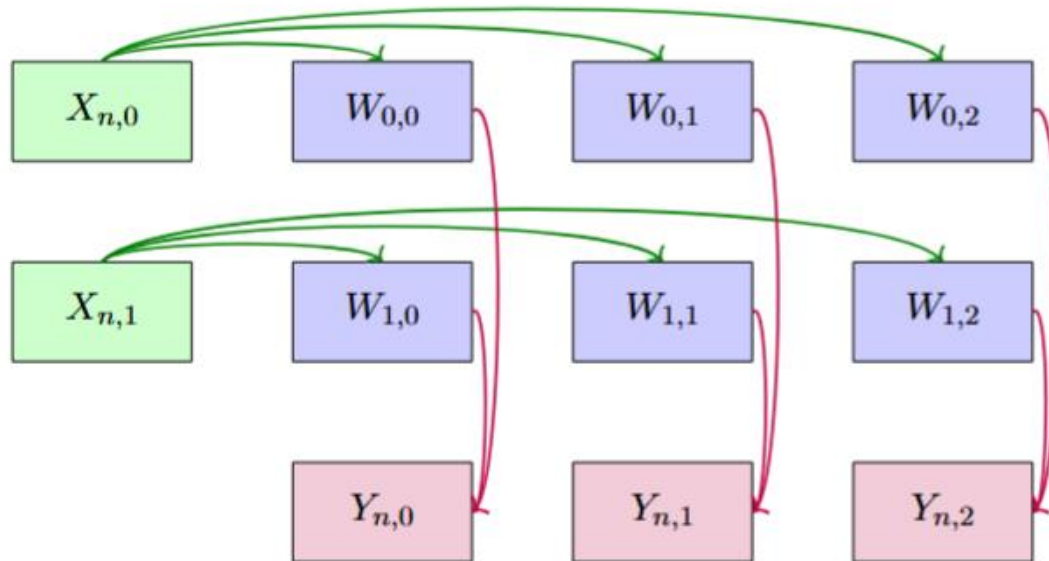
- ❑ Matrix Multiplications
- ❑ Hardware Implementation via Broadcast
-  Systolic Arrays
- ❑ Tiling for Large Matrices

Baseline Implementation Layout



- Consider the layout of baseline multiplier
- Row buffer for X
- Row buffer for Y
- Matrix W

Routing Challenges: Fanout

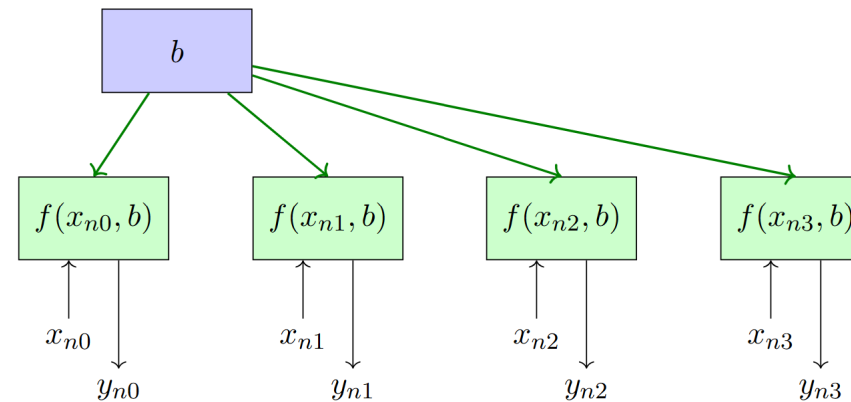


- ❑ Each X_{ni} must be **broadcast**
 - To M locations
- ❑ We say the **fanout** of X_{nj} is M
- ❑ High fanout is hard to route
- ❑ Creates large number of wires
- ❑ Many wires can be long
- ❑ Difficult to meet timing

Simple Example

```
for (int i = 0; i < N; i++) {  
  #pragma HLS PIPELINE II=1  
  for (int j = 0; j < M; j++) {  
    #pragma HLS UNROLL  
    y[i][j] = f(x[i][j], b);  
  }  
}
```

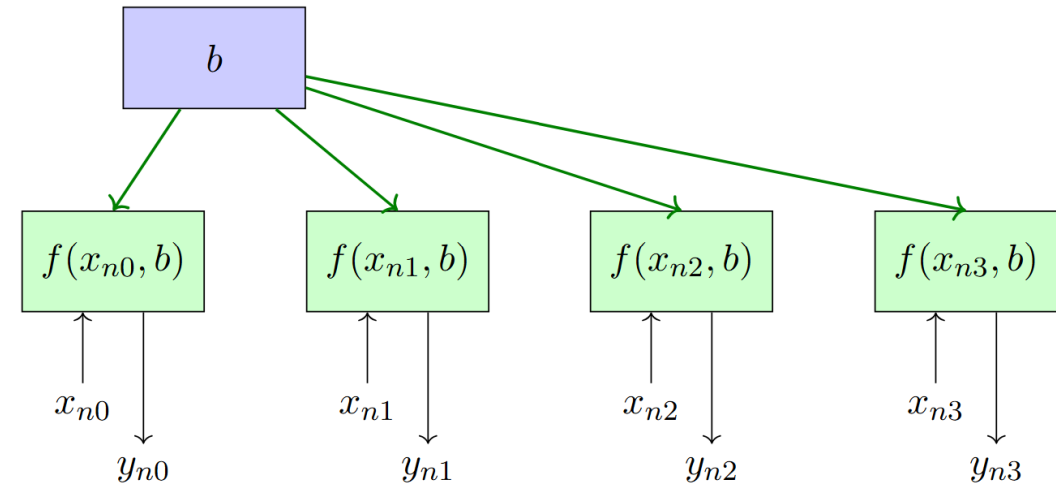
- Consider simple example to left
- Parameter b is broadcast
 - To M instances of function $f(x_{ij}, b)$
- Fanout is M



Fanout Visualized

Fanout is M

```
for (int i = 0; i < N; i++) {  
  #pragma HLS PIPELINE II=1  
  for (int j = 0; j < M; j++) {  
    #pragma HLS UNROLL  
    y[i][j] = f(x[i][j], b);  
  }  
}
```



Reduced Fan-Out Design

```
bcopy[0] = b;
for (int j=1; j < M; j++) {
    bcopy[j] = bcopy[j-1];
}

for (int i = 0; i < N; i++) {
    #pragma HLS PIPELINE II=1
    for (int j = 0; j < M; j++) {
        #pragma HLS UNROLL
        y[i][j] = f(x[i][j], bcopy[j]);
    }
}
```

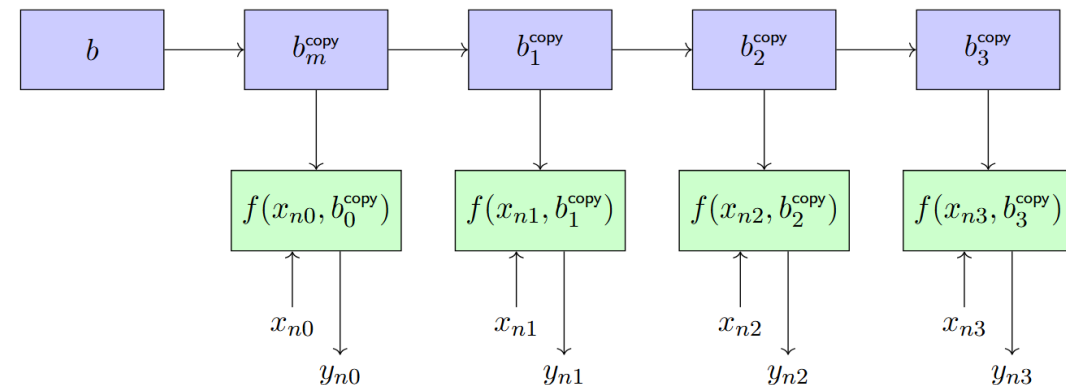
- ❑ Improved design with lower fanout
- ❑ Copy b in a shift register
 - Takes M cycles
 - And M additional registers
- ❑ Use a delay line
 - Shift in one element of $bcopy$ at a time
- ❑ Use copy of b in each function

Reduced Fan-Out Visualized

```
bcopy[0] = b;
for (int j=1; j < M; j++) {
    bcopy[j] = bcopy[j-1];
}

for (int i = 0; i < N; i++) {
    #pragma HLS PIPELINE II=1
    for (int j = 0; j < M; j++) {
        #pragma HLS UNROLL
        y[i][j] = f(x[i][j], bcopy[j]);
    }
}
```

- ❑ Fan-out is reduced from M to 2
- ❑ Computations can be local
- ❑ Key lesson: Add additional delay and register
- ❑ But greatly improve routing

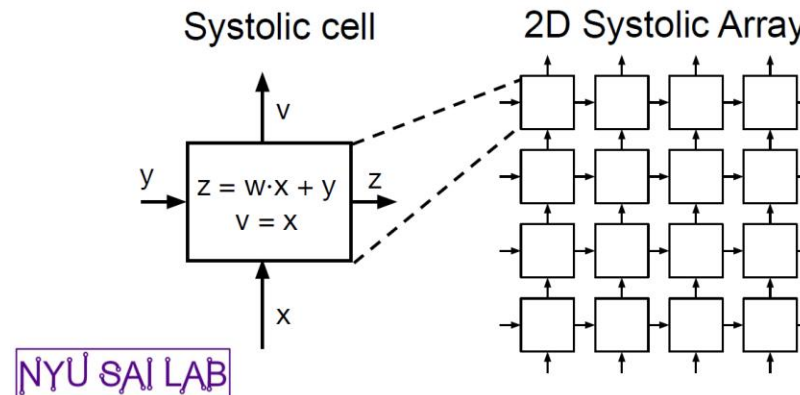


Systolic Arrays

- ❑ See slides from Prof Sai Zhang
- ❑ Provided to NYU students on Brightspace

Systolic Array (Weight Stationary Version)

- Kung and Leiserson, "Systolic Arrays for VLSI," 1978 and Kung, "Why systolic architectures?" 1982
- 2D grid of multiplier-accumulators (MACs) for matrix multiplication
- Used by Google TPU for deep learning (2017), etc



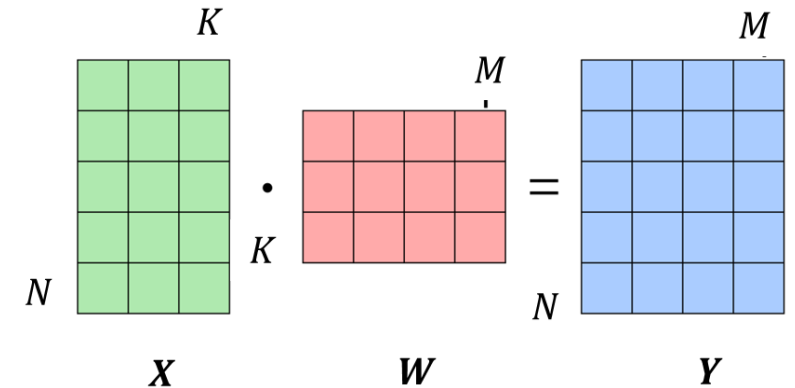
TPU (Google)

Outline

- ❑ Matrix Multiplications
- ❑ Hardware Implementation via Broadcast
- ❑ Systolic Arrays
-  Tiling

Memory Bottleneck

- ❑ Suppose matrices X and Y can only be read one word per cycle
- ❑ Reading and writing take $NK + NM$ cycles
- ❑ If we have $K \times M$ systolic array:
 - Computation takes $O(N)$ cycles
- ❑ So, memory, not computation is the bottleneck



Block Matrices

□ Partition a matrix P into sub-matrices or blocks

Full matrix

$$\mathbf{P} = \left[\begin{array}{cc|cc} 1 & 2 & 2 & 7 \\ 1 & 5 & 6 & 2 \\ \hline 3 & 3 & 4 & 5 \\ 3 & 3 & 6 & 7 \end{array} \right].$$

Matrix blocks

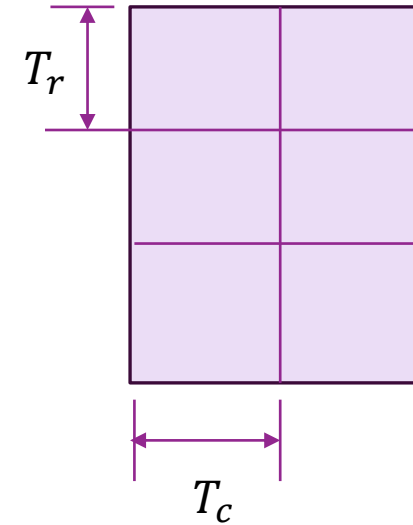
$$\begin{aligned} \mathbf{P}_{11} &= \begin{bmatrix} 1 & 2 \\ 1 & 5 \end{bmatrix}, & \mathbf{P}_{12} &= \begin{bmatrix} 2 & 7 \\ 6 & 2 \end{bmatrix}, \\ \mathbf{P}_{21} &= \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}, & \mathbf{P}_{22} &= \begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix}. \end{aligned}$$

Block matrix form

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_{11} & \mathbf{P}_{12} \\ \mathbf{P}_{21} & \mathbf{P}_{22} \end{bmatrix}.$$

Tiling for Matrix Multiplication

- Divide rows in tiles of size T_r
 - X and Y will have $\frac{N}{T_r}$ block rows
 - W will have $\frac{K}{T_r}$ block rows
- Divide columns in tiles of size T_c
 - Y and W will have $\frac{M}{T_c}$ block columns
 - X will have $\frac{K}{T_c}$ block columns
- Systolic array will be $T_r \times T_c$



Tiling Procedure

- ❑ Load full matrix $W = [W_{kj}]$ into local memory once
 - Let $W_{kj} = T_r \times T_r$ block
- ❑ For each block row i
 - Load T_r rows of X_i and Y into local memory
 - Perform matrix multiplications $X_{ik}W_{kj}$ via systolic arrays
 - Performed for $k = 1, \dots, \frac{K}{T_c}$ and $j = 1, \dots, \frac{M}{T_r}$
 - Complexity of each block matrix multiplication is N_r
 - So, cost is $\frac{N_r KM}{T_c T_r}$

